Lab 5 — Assignment 2 Report: DynamoDB Notes App for the Lab 4 Matomo App

Course: Cloud Computing, T.E. Computer, Sem V (2026-2027) | **Account**: calista-labs (386347568553) | **Region**: ap-south-1

EC2: matomo-lab4 (i-077933894a1f47c75), app at http://3.111.58.245:8081/ | **DynamoDB**: matomo-site-notes | **Date**: 2026-09-22

Architecture

Browser -> HTTP:8081 -> EC2 (matomo-lab4: Nginx + dedicated PHP-FPM pool + AWS SDK for PHP) -> IAM role credentials via instance metadata -> DynamoDB table matomo-site-notes. IAM role matomo-dynamodb-role is attached directly to the EC2 instance and scoped by a customer-managed policy to exactly this one table's ARN.

Lab 4's app (Matomo) uses MariaDB (SQL) — Assignment 1 covered Case A. This assignment is Case B: the opposite database type, NoSQL. Since Matomo has no natural use for a NoSQL store, a small standalone PHP app ("site operational notes") was deployed alongside it on the same instance, on a separate port and PHP-FPM pool, so the working Assignment 1 deployment was never touched.

Database & security settings

Setting	Value
Table	matomo-site-notes, on-demand capacity
Keys	Partition: site_id (Number) — Sort: note_id (String)
IAM role	matomo-dynamodb-role, trusted entity EC2
IAM policy	matomo-dynamodb-policy — scoped to this table's ARN only (Get/Put/Update/Delete/Query/Scan)
Credentials in code	None — SDK default credential provider resolves to the instance's IAM role via IMDSv2
App exposure	Port 8081, restricted to one admin IP at both the Security Group and the host ufw firewall

Deployment & verification

Table created via the console. IAM policy and role created while signed in as the account root user — a one-time requirement, since the calista-labs IAM user deliberately holds only IAMReadOnlyAccess. Role attached live to the running instance (no restart). Confirmed the instance could see its own role via `curl` against the IMDSv2 metadata endpoint before writing any app code.

AWS SDK for PHP installed via Composer in a fresh `/var/www/dynamo-notes` directory. App given its own Nginx site and PHP-FPM pool listening on port 8081, kept fully separate from Matomo's config on port 80.

Gotcha resolved: port 8081 was unreachable externally even after the Security Group rule was added, because this account's instances enforce a second firewall layer (`ufw`) that had never been told about port 8081 — only ports 22/80/443 were allowed there from the original Lab 4 hardening. Fixed with `sudo ufw allow 8081/tcp`.

CRUD evidence (all via the running app at :8081)

Op	Action	Result
Create	Added note "RDS migration verified" for site 1	New item in matomo-site-notes
Read	Page reload	Note listed — "Notes (1)"
Update	Clicked "Mark resolved"	resolved: false -> true
Delete	Clicked "Delete", confirmed	Item removed — "Notes (0)"

Full screenshots for every step (table creation, IAM policy, IAM role, EC2 attachment, live app, all 4 CRUD ops, and the item detail view) are in `Lab5-Assignment2-Screenshots.docx`.

Attribute-type evidence

The created item carries 5 distinct DynamoDB types, confirmed directly in the console's item detail view:

Attribute	Type
site_id, created_at	Number (N)
note_id, title, body	String (S)
resolved	Boolean (BOOL)
tags	List (L)
meta	Map (M)

Requirement checklist

☑ Create a DynamoDB table with a partition key (site_id) and sort key (note_id)

☑ Attach an IAM role to EC2 for DynamoDB access — no hardcoded AWS keys

☑ Connect to the Lab 4 EC2 app — new standalone app deployed on the same instance

☑ Demonstrate all CRUD from the running app

☑ At least 5 distinct DynamoDB attribute types present on a stored item (N, S, BOOL, L, M)

☑ DynamoDB access restricted to the EC2 IAM role only